

Путь Python-разработчика

От полного новичка до первой работы

Владислав

7 августа 2026

Содержание

1	Путь Python-разработчика	5
I	2 Том I. Основы программирования	6
	Быстрые ссылки	7
2.1	Как думает компьютер	8
	После этой главы вы сможете	8
	Вы уже сталкивались с программированием	9
	Задача, алгоритм и программа	10
	Компьютер не умеет догадываться	11
	Первая практика	12
	Короткий итог	12
2.2	Почему компьютер понимает только 0 и 1	14
	Почему компьютер не понимает человеческий язык	14
	Что находится внутри компьютера	15
	Почему именно два состояния	17
	Откуда появились 0 и 1	18
	Что такое бит	20
	Как из двух состояний получается сложная информация	21
	Всё превращается в числа	22
	Что происходит технически	24
	От транзисторов к логическим операциям	25
	Немного истории	26
	Почему это важно будущему Python-разработчику	27
	Практика	28
	Задание 1	28
	Задание 2	28
	Задание 3	29
	Проверьте себя	29
	Главное, что нужно запомнить	29
	Что дальше	30
2.3	Как компьютер хранит данные	31

Бит — самая маленькая единица информации	32
Байт	33
Как компьютер понимает, что хранится в байте?	35
Как хранится текст	37
Почему этого оказалось недостаточно	39
Как хранится число	40
Как хранится изображение	41
Как хранится музыка	43
Как хранится программа	44
Главное, что нужно запомнить	46
Что дальше?	47
2.4 Как запускаются программы	48
После этой главы вы сможете	48
Файл с кодом	48
Интерпретатор	48
Процесс	49
Короткий итог	49
Практика	49
2.5 Зачем нужны языки программирования	50
После этой главы вы сможете	50
Язык как мост	50
Синтаксис	50
Почему языков много	51
Короткий итог	51
Практика	51
2.6 Почему Python	52
После этой главы вы сможете	52
Читаемость	52
Где используется Python	52
Ограничения	53
Короткий итог	53
Практика	53
2.7 Рабочее пространство Python	54
После этой главы вы сможете	54
Основные инструменты	54

Папка проекта	54
Терминал	55
Виртуальное окружение	55
Короткий итог	55
Практика	55
2.8 Первая Python-программа	56
После этой главы вы сможете	56
Создаем файл	56
Пишем код	56
Запускаем программу	57
Если команда не сработала	57
Короткий итог	57
Практика	57

1 Путь Python-разработчика

От полного новичка до первой работы

Практическая книга по Python и backend-разработке: от первых шагов до уверенной подготовки к первой работе.

Эта книга строится вокруг простого принципа: сначала мышление разработчика, затем синтаксис, инструменты и реальные проекты.

Вы будете постепенно разбирать задачи, превращать их в алгоритмы, писать понятный Python-код и собирать профессиональную рабочую среду.

i Как читать книгу

Двигайтесь по главам последовательно. В начале важнее понимать ход мысли, чем запоминать команды.

Часть I

2 Том I. Основы программирования

В этом томе собраны базовые главы, которые помогают понять, как компьютер работает с инструкциями, данными и первыми программами.

Быстрые ссылки

- [2.1 Как думает компьютер](#)
- [2.2 Почему компьютер понимает только 0 и 1](#)
- [2.3 Как компьютер хранит данные](#)
- [2.4 Как запускаются программы](#)
- [2.5 Зачем нужны языки программирования](#)
- [2.6 Почему Python](#)
- [2.7 Рабочее пространство Python](#)
- [2.8 Первая Python-программа](#)

i Как читать этот том

Идите по главам последовательно. Сначала разберитесь, как компьютер воспринимает инструкции и данные, затем переходите к рабочему пространству и первой программе на Python.

2.1 Как думает компьютер

Прежде чем изучать переменные, условия, циклы и функции, полезно разобраться с более фундаментальным вопросом:

что вообще происходит, когда человек пишет программу?

На первый взгляд кажется, что программирование начинается с изучения языка. Например, с Python.

Но язык — это только инструмент.

Настоящее программирование начинается раньше: с умения сформулировать задачу и разбить её на последовательность понятных действий.

В этой главе мы разберёмся, что такое программа, почему компьютер не умеет догадываться, зачем существуют языки программирования и какую роль во всём этом играет Python.

После этой главы вы сможете

После изучения главы вы сможете:

- объяснить своими словами, что такое программа;
- понимать разницу между задачей, алгоритмом и кодом;
- объяснить, почему компьютеру нужны точные инструкции;
- понимать, зачем существуют языки программирования;
- объяснить на базовом уровне, что происходит после запуска Python-программы;
- разбивать простую бытовую задачу на последовательность шагов.

i Главная цель главы

Пока не нужно запоминать команды Python.

Сначала необходимо понять принцип: **компьютер выполняет инструкции, а разработчик эти инструкции проектирует.**

Вы уже сталкивались с программированием

Представим простую задачу:

приготовить чашку чая.

Для человека этой фразы обычно достаточно. Мы автоматически понимаем, что нужно сделать.

Налить воду. Включить чайник. Подготовить кружку. Положить чай. Дождаться кипения воды. Налить воду в кружку.

Большую часть этих действий мы даже не проговариваем.

Наш мозг самостоятельно заполняет пропущенные шаги.

Но теперь представим устройство, которое умеет выполнять команды, но совершенно не умеет догадываться.

Мы говорим ему:

Приготовь чай.

Для человека инструкция понятна.

Для машины — практически бесполезна.

Нужно описать задачу гораздо точнее:

1. Найди чайник.
2. Проверь, есть ли в нём вода.
3. Если воды недостаточно — добавь её.
4. Подключи чайник к электропитанию.
5. Включи чайник.
6. Возьми кружку.
7. Положи в неё чай.
8. Дождись кипения воды.
9. Налей горячую воду в кружку.
10. Подожди несколько минут.

Теперь большая задача превратилась в последовательность небольших действий.

Именно с этого начинается программирование.

! Запомните

Программирование — это прежде всего умение разбивать задачу на понятные и точные шаги.

Язык программирования нужен уже после того, как мы понимаем, какие действия должен выполнить компьютер.

Задача, алгоритм и программа

На этом этапе важно различать три понятия:

задача — результат, который мы хотим получить;

алгоритм — последовательность действий для достижения результата;

программа — алгоритм, записанный в форме, которую компьютер способен выполнить.

Возьмём простой пример.

Наша задача:

узнать остаток денег после ежемесячных расходов.

Алгоритм можно описать обычным языком:

1. Узнать месячный доход.
2. Узнать месячные расходы.
3. Вычесть расходы из дохода.
4. Показать результат.

А позже мы сможем записать этот алгоритм на Python:

```
monthly_income = 5000
monthly_expenses = 3200

balance = monthly_income - monthly_expenses

print(balance)
```

Сейчас не нужно разбираться в каждой строке.

Важно увидеть связь:

```
Задача
  ↓
Алгоритм
  ↓
Код
  ↓
Выполнение
  ↓
Результат
```

Код не появляется из ниоткуда.

Перед кодом почти всегда должно существовать понимание того, **что именно должна делать программа.**

Компьютер не умеет догадываться

Одна из важнейших мыслей для начинающего разработчика:

компьютер не понимает намерения программиста.

Он выполняет то, что написано.

Не то, что мы хотели написать.

Не то, что имели в виду.

И не то, что кажется очевидным человеку.

Представим инструкцию:

Если пользователь взрослый, разрешить регистрацию.

Человек понимает общий смысл.

Но компьютеру этого недостаточно.

Что значит «взрослый»?

18 лет?

21 год?

Возраст должен быть больше 18 или больше либо равен 18?

Что делать, если возраст вообще не указан?

Что делать, если вместо возраста пользователь ввёл слово?

Каждый подобный вопрос разработчику приходится превращать в конкретные правила.

Например:

```
Если возраст пользователя больше или равен 18:  
    разрешить регистрацию.  
Иначе:  
    отказать в регистрации.
```

Позже это превратится в настоящий Python-код.

💡 Как думает разработчик

Начинающий программист часто спрашивает:

Какую команду Python мне здесь написать?

Более полезный вопрос:

Какие точные действия должна выполнить программа?

Сначала решение. Затем код.

Первая практика

Пока не открывайте Python.

Попробуйте выполнить упражнение обычным текстом.

Нужно написать инструкцию для робота, который должен почистить зубы.

Плохой вариант:

1. Взять щётку.
2. Почистить зубы.

Слишком много действий скрыто внутри второго шага.

Попробуйте описать процесс подробнее.

Например, подумайте:

- откуда взять зубную щётку;
- что сделать с зубной пастой;
- сколько пасты использовать;
- когда открыть воду;
- когда её закрыть;
- сколько времени чистить зубы;
- что сделать со щёткой после использования.

Чем точнее инструкция, тем ближе она к алгоритму.

Типичная ошибка новичка

Не пытайтесь сразу переводить любую задачу в Python.

Сначала научитесь формулировать решение обычными словами.

Если алгоритм непонятен на русском языке, код почти наверняка тоже получится запутанным.

Короткий итог

На этом этапе достаточно запомнить четыре вещи:

1. У программы всегда есть задача.

2. Задачу необходимо разбить на последовательность действий.
3. Компьютер выполняет инструкции буквально.
4. Python нужен для записи этих инструкций в форме, которую можно выполнить.

В следующем разделе мы разберёмся, **почему компьютер вообще способен работать только с очень простыми сигналами и откуда появились знаменитые 0 и 1.**

2.2 Почему компьютер понимает только 0 и 1

В предыдущей главе мы разобрались, что компьютер выполняет только точные инструкции.

Но возникает следующий вопрос:

почему компьютер вообще работает с нулями и единицами?

Почему нельзя заставить его сразу понимать слова, числа или команды так, как это делает человек?

Чтобы ответить на этот вопрос, нужно немного заглянуть внутрь компьютера.

Не слишком глубоко. Нам не понадобится изучать электронику или физику полупроводников.

Для Python-разработчика важно понять только основной принцип.

Почему компьютер не понимает человеческий язык

Человек умеет использовать контекст.

Если сказать:

Сделай чай.

другой человек обычно понимает, что нужно сделать.

Он знает:

- где находится чайник;
- сколько примерно налить воды;
- какую кружку взять;
- когда вода закипела;
- что означает слово «чай».

Большая часть информации вообще не проговаривается.

Наш мозг автоматически достраивает недостающие шаги.

Компьютер так не умеет.

Если программа должна выполнить определённое действие, это действие необходимо описать достаточно точно.

Компьютер не понимает намерение разработчика.

Он выполняет только то, что действительно указано в программе.

! Главная мысль

Компьютер не обладает здравым смыслом.

Он не догадывается, что программист хотел сделать.

Он выполняет конкретные инструкции.

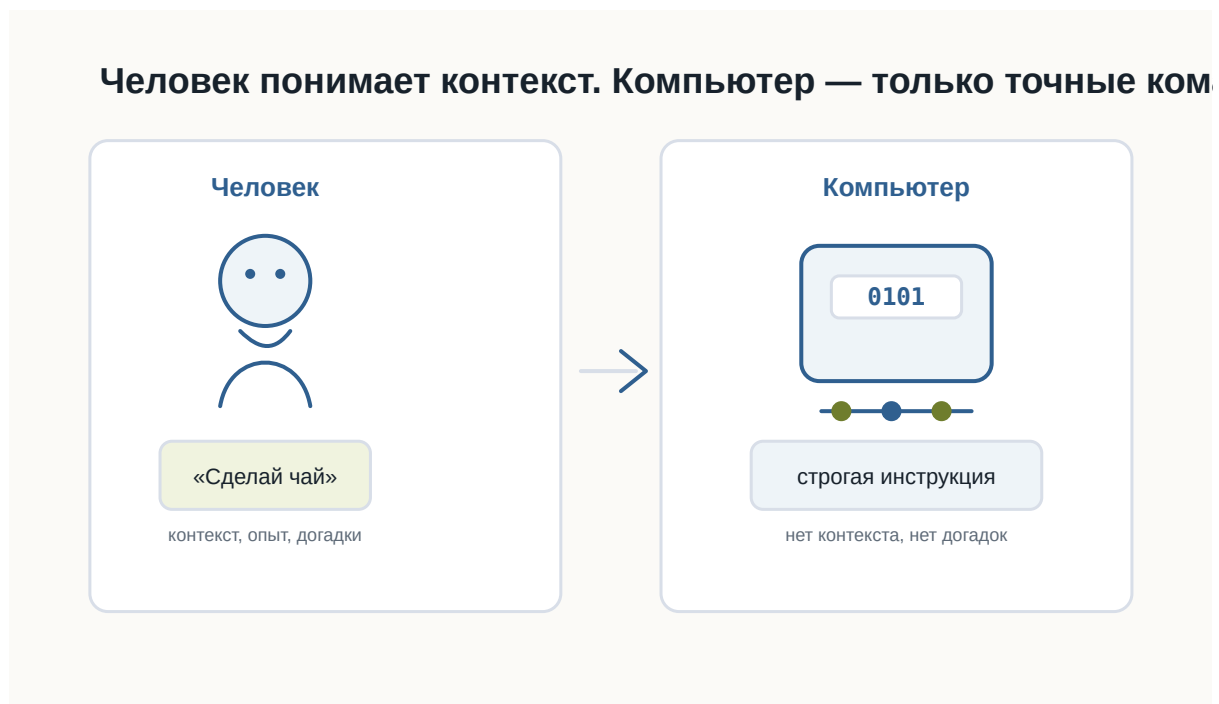


Рисунок 1.1: Человек использует контекст, а компьютеру нужны точные инструкции

Что находится внутри компьютера

Современный компьютер состоит из огромного количества электронных компонентов.

Один из самых важных — **транзистор**.

Транзистор — это очень маленький электронный переключатель.

Для понимания программирования его можно представить как обычный выключатель света.

Выключатель имеет два устойчивых состояния:

Включён

Выключен

Транзистор работает по похожему принципу.

В упрощённом виде для нас достаточно считать, что он различает два состояния электрического сигнала:

Высокий уровень сигнала

Низкий уровень сигнала

Или ещё проще:

Есть сигнал

Нет сигнала

i Насколько точно это объяснение?

В реальной электронике всё немного сложнее.

Компьютер работает не с буквальными состояниями «ток есть» и «тока нет», а с диапазонами напряжений, которые электронные схемы интерпретируют как два логических состояния.

Но для первого знакомства модель «включено / выключено» достаточно точно передаёт основной принцип.

Современный процессор содержит огромное количество транзисторов.

Каждый из них очень мал, но вместе они способны выполнять миллиарды операций.

Чем больше транзисторов можно разместить внутри микросхемы и чем быстрее они переключаются, тем более сложные вычисления способен выполнять процессор.



Рисунок 1.2: Транзистор как электронный переключатель

Почему именно два состояния

Можно задать вполне разумный вопрос:

Почему только два?

Почему не три?

Почему не десять?

Теоретически вычислительную систему можно построить и на другом количестве состояний.

Такие идеи действительно существовали и существуют.

Но двоичная система оказалась чрезвычайно удобной для электронной техники.

Причина — **надёжность**.

Представим, что электронная схема должна различать десять уровней напряжения.

Условно:

0.5 В
1.0 В
1.5 В
2.0 В
...

Тогда даже небольшая помеха может привести к ошибке.

Например, устройство ожидало 1.5 В, а из-за электрического шума получило 1.6 В.

Какое это состояние?

Чем больше состояний должна различать система, тем сложнее надёжно определить каждое из них.

С двумя состояниями гораздо проще.

Электронная схема может работать по принципу:

Низкий уровень → одно состояние

Высокий уровень → другое состояние

Между ними можно оставить запас.

Если сигнал немного изменится из-за помех, схема всё равно правильно определит его состояние.

Именно поэтому два состояния особенно хорошо подходят для быстрых и надёжных цифровых устройств.

💡 Простая аналогия

Представьте два выключателя.

Первый имеет только два положения:

ВКЛ

ВЫКЛ

Второй имеет десять очень близко расположенных положений.

Каким выключателем проще пользоваться без ошибки?

Примерно по той же причине электронным схемам удобно работать с двумя состояниями.

Откуда появились 0 и 1

Сами транзисторы не знают никаких цифр.

Внутри компьютера не бегают настоящие нули и единицы.

Цифры **0** и **1** придумали люди как удобные обозначения двух состояний.

Условно можно записать:

Низкий уровень сигнала → 0

Высокий уровень сигнала → 1

Можно было выбрать и другие обозначения:

НЕТ / ДА

ВЫКЛ / ВКЛ

FALSE / TRUE

ЧЁРНЫЙ / БЕЛЫЙ

Принцип работы компьютера от этого бы не изменился.

Но обозначения 0 и 1 оказались особенно удобными, потому что позволяют использовать **двоичную систему счисления**.

О ней мы поговорим подробнее позже.

Пока достаточно понимать:

0 и 1 — это математические обозначения двух физических состояний.

Восемь включателей можно записать как восемь битов



Рисунок 1.3: Включённые и выключенные состояния можно записать как последовательность нулей и единиц

Что такое бит

Теперь можно аккуратно познакомиться с первым важным термином.

Бит — это минимальная единица цифровой информации, которая может принимать одно из двух значений:

0

или

1

Слово bit произошло от английского выражения:

binary digit — двоичная цифра.

Один бит способен хранить только два варианта.

Например:

0 → нет

1 → да

или:

0 → свет выключен

1 → свет включён

Но одного бита недостаточно для хранения сложной информации.

Поэтому компьютеры используют большие последовательности битов.

Например:

01000001

Здесь уже восемь битов.

Группа из восьми битов называется **байтом**.

Подробно с битами, байтами и тем, как из них складываются данные, мы познакомимся в следующей главе.

i Не нужно зубрить

Пока достаточно запомнить:

- бит принимает значение 0 или 1;
- несколько битов можно объединять;
- из больших последовательностей битов компьютер строит сложные данные.

Как из двух состояний получается сложная информация

На первый взгляд кажется странным:

как всего два состояния могут описать фотографию, музыку или целую компьютерную игру?

Ответ заключается в количестве комбинаций.

Возьмём один бит:

0
1

Возможно только **2 комбинации**.

Теперь два бита:

00
01
10
11

Уже **4 комбинации**.

Три бита:

000
001
010
011
100
101
110
111

Получаем **8 комбинаций**.

Чем больше битов, тем больше разных состояний можно закодировать.

Для n битов количество комбинаций равно:

$$2^n$$

Например:

8 бит → 256 комбинаций

Именно поэтому даже небольшая последовательность битов уже может представлять достаточно много разных значений.

Компьютер использует огромные последовательности битов.

Поэтому двух базовых состояний оказывается достаточно для хранения практически любой цифровой информации.

Всё превращается в числа

Компьютер не хранит букву так, как её видит человек.

Возьмём:

A

Компьютеру нужно договориться, какое число будет обозначать эту букву.

Например, в одной из распространённых систем кодирования латинской букве A соответствует число:

65

Число 65, в свою очередь, можно представить в двоичной форме:

01000001

Получается цепочка:

A

↓

65

↓

01000001

С изображениями идея похожая.

Фотография состоит из пикселей.

Каждый пиксель описывается числами.

Например, числа могут определять интенсивность красного, зелёного и синего цветов.

Дальше эти числа снова представляются последовательностями битов.

Фотография

↓

Пиксели

↓

Числа

↓

0 и 1

Музыка также может быть представлена числами.

Звуковая волна измеряется много раз в секунду.

Результаты измерений записываются как числа.

Звук

↓

Измерения

↓

Числа

↓

0 и 1

Видео можно рассматривать как последовательность изображений вместе со звуком.

Программы тоже состоят из данных и инструкций, которые в конечном итоге представлены тем же способом.



Рисунок 1.4: Текст, изображения, звук и программы в конечном итоге превращаются в цифровые данные

! Главная мысль

Компьютер способен работать с текстом, фотографиями, музыкой и программами не потому, что понимает их смысл. Он умеет представлять всё это числами. А числа можно представить последовательностями 0 и 1.

Что происходит технически

Теперь немного точнее посмотрим на путь информации.

Когда программа работает с некоторым значением, происходит несколько уровней преобразований.

Упрощённо:

Данные программы
↓
Числовое представление
↓
Биты
↓
Электрические сигналы
↓
Электронные схемы

Например, Python-программа может работать со строкой:

```
message = "Hello"
```

Для разработчика это текст.

Python рассматривает его как объект строки.

Дальше компьютерное оборудование работает уже с машинным представлением этих данных.

В памяти находятся не слова в человеческом смысле, а закодированные значения.

На самом нижнем уровне они физически представлены состояниями электронных элементов.

Важно понимать, что программист почти никогда не работает напрямую с этим уровнем.

Python специально скрывает большую часть сложности.

Мы можем писать:

```
message = "Hello"
```

и не думать о том, какие конкретно биты находятся в оперативной памяти.

Это один из главных принципов программирования:

каждый следующий уровень скрывает сложность предыдущего.

Мы ещё много раз встретим эту идею в книге.

От транзисторов к логическим операциям

Один транзистор сам по себе умеет немного.

Но транзисторы объединяют в электронные схемы.

Из них можно создавать так называемые **логические элементы**.

Они выполняют простые операции над двумя состояниями.

Например:

```
И  
ИЛИ  
НЕ
```

Позже из таких простых операций строятся более сложные схемы:

логические элементы



сумматоры



регистры



память



процессор

В результате миллиарды очень простых переключателей способны совместно выполнять сложные программы.

i Почему это важно программисту

Python-разработчику не нужно уметь проектировать процессоры. Но полезно понимать, что любой сложный код в конечном итоге выполняется системой, построенной из огромного количества простых логических операций. Это помогает избавиться от ощущения, что компьютер работает благодаря какой-то магии.

Немного истории

Первые электронные компьютеры выглядели совсем не так, как современные ноутбуки.

До появления транзисторов в вычислительной технике широко использовались **электронные лампы**.

Они выполняли похожую функцию — позволяли управлять электрическими сигналами.

Но у них были серьёзные недостатки.

Лампы были:

- большими;
- горячими;
- энергозатратными;
- сравнительно ненадёжными.

Ранние компьютеры могли занимать целые помещения.

Появление транзистора позволило резко уменьшить размеры электронных схем.

Транзисторы были значительно меньше, экономичнее и надёжнее ламп.

Позже инженеры научились размещать множество транзисторов на одной микросхеме.

Количество транзисторов продолжало расти.

Сегодня внутри одного современного процессора могут находиться **миллиарды транзисторов**.

То, для чего когда-то требовалось целое помещение, теперь помещается в небольшом устройстве.



Рисунок 1.5: Упрощённая эволюция вычислительной техники от электронных ламп до современных процессоров

Почему это важно будущему Python-разработчику

Можно задать вопрос:

Если Python скрывает все эти детали, зачем мне вообще знать про транзисторы и биты?

Чтобы писать первые программы, эти знания действительно не обязательны.

Но они помогают сформировать правильное представление о компьютере.

Когда позже мы начнём говорить о:

- переменных;
- типах данных;

- памяти;
- файлах;
- кодировках;
- базах данных;
- сетях;

вы будете понимать, что всё это разные способы организации одной и той же цифровой информации.

Кроме того, становится понятнее важный принцип:

абстракции позволяют программисту работать со сложной системой, не думая постоянно о её нижних уровнях.

Python — один из таких уровней абстракции.

Практика

Задание 1

Представьте четыре выключателя.

Каждый может быть либо включён, либо выключен.

Попробуйте самостоятельно записать все возможные комбинации из четырёх состояний с помощью 0 и 1.

Подсказка:

```
0000
0001
0010
...
```

Сколько комбинаций получится?

Задание 2

Попробуйте объяснить своими словами:

Почему два состояния удобнее для электронной системы, чем десять?

Не используйте определение из книги дословно.

Главная задача — сформулировать мысль самостоятельно.

Задание 3

Выберите любой объект:

- фотографию;
- песню;
- текстовый документ;
- видео.

Попробуйте мысленно пройти путь:

Объект

↓

Как его можно представить числами?

↓

Как числа можно представить битами?

Не нужно знать точные форматы файлов.

Нас интересует только общий принцип.

Проверьте себя

Попробуйте ответить без подсказок:

1. Что такое транзистор в упрощённом представлении?
2. Почему электронным схемам удобно использовать два состояния?
3. Действительно ли внутри компьютера находятся цифры 0 и 1?
4. Что такое бит?
5. Сколько возможных состояний имеет один бит?
6. Почему несколько битов позволяют хранить больше информации?
7. Как текст можно превратить в двоичные данные?
8. Можно ли по одному принципу хранить текст, музыку и фотографии?
9. Зачем Python-разработчику понимать эти основы, если Python скрывает работу электроники?

Если вы можете уверенно объяснить ответы своими словами, значит основная идея главы усвоена.

Главное, что нужно запомнить

1. В основе цифрового компьютера лежат электронные элементы с двумя различимыми состояниями.
2. Эти состояния условно обозначают как 0 и 1.
3. Один 0 или 1 представляет один бит информации.

4. Последовательности битов позволяют кодировать множество различных значений.
5. Текст, изображения, музыка, видео и программы в конечном итоге представляются цифровыми данными.
6. Python скрывает большую часть нижнего уровня и позволяет работать с понятными человеку объектами.

! Главный вывод главы

Компьютер не «говорит на языке нулей и единиц».

Нули и единицы — это удобная математическая модель двух физических состояний электронных схем.

Комбинируя огромное количество таких простых состояний, компьютер способен хранить данные и выполнять сложнейшие программы.

Что дальше

Теперь мы знаем, почему компьютер использует два состояния и откуда появляются 0 и 1.

Но остаётся важный вопрос:

как именно из последовательностей битов получаются числа, буквы, изображения и другие данные?

В следующей главе мы разберёмся, как компьютер хранит информацию и познакомимся с битами, байтами и кодированием данных подробнее.

2.3 Как компьютер хранит данные

! Важное уведомление

Главный вопрос главы:

Если компьютер знает только 0 и 1, как он хранит текст, числа, фотографии, музыку и программы?

После предыдущей главы мы уже знаем важную вещь:

Любая информация внутри компьютера состоит только из нулей и единиц.

Но возникает следующий вопрос.

Когда вы открываете фотографию, запускаете игру или читаете текстовый файл, вы не видите последовательность вроде

010011010100101011...

Вы видите изображение, буквы, числа и кнопки.

Почему?

Потому что сами биты ничего не означают.

Смысл появляется только тогда, когда существует правило их интерпретации.

Бит — самая маленькая единица информации

Бит может принимать только два значения.

0

или

1

Это похоже на обычный выключатель.

Выключено -> 0

Включено -> 1

Одного бита почти никогда недостаточно.

Поэтому биты объединяют в группы.

Байт

Самая известная группа — **байт**.

Один байт состоит из восьми битов.

10100110

или

00000000

или

11111111

Всего существует

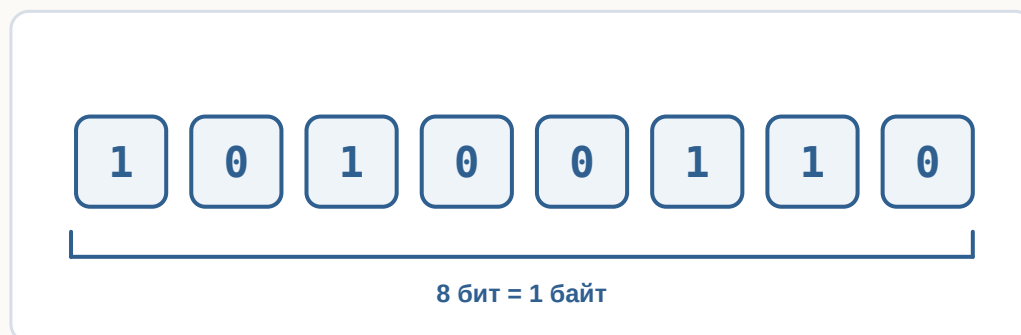
$$2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 256$$

различных комбинаций.

Поэтому один байт способен хранить одно из **256 различных значений**.

Это очень удобно.

Байт — это группа из восьми битов



возможных вариантов
 $2^8 = 256$ комбинаций

Рисунок 1.6: Восемь битов образуют один байт.

Как компьютер понимает, что хранится в байте?

Представьте число

65

Что это означает?

Без контекста — ничего.

Это может быть:

- возраст;
- скорость;
- номер страницы;
- температура.

То же самое происходит с байтами.

Последовательность

01000001

сама по себе ничего не означает.

Но если договориться считать её символом ASCII, получится буква

A

Если считать её числом —

65

То есть одинаковые биты могут означать разные вещи.

Все зависит от правил.

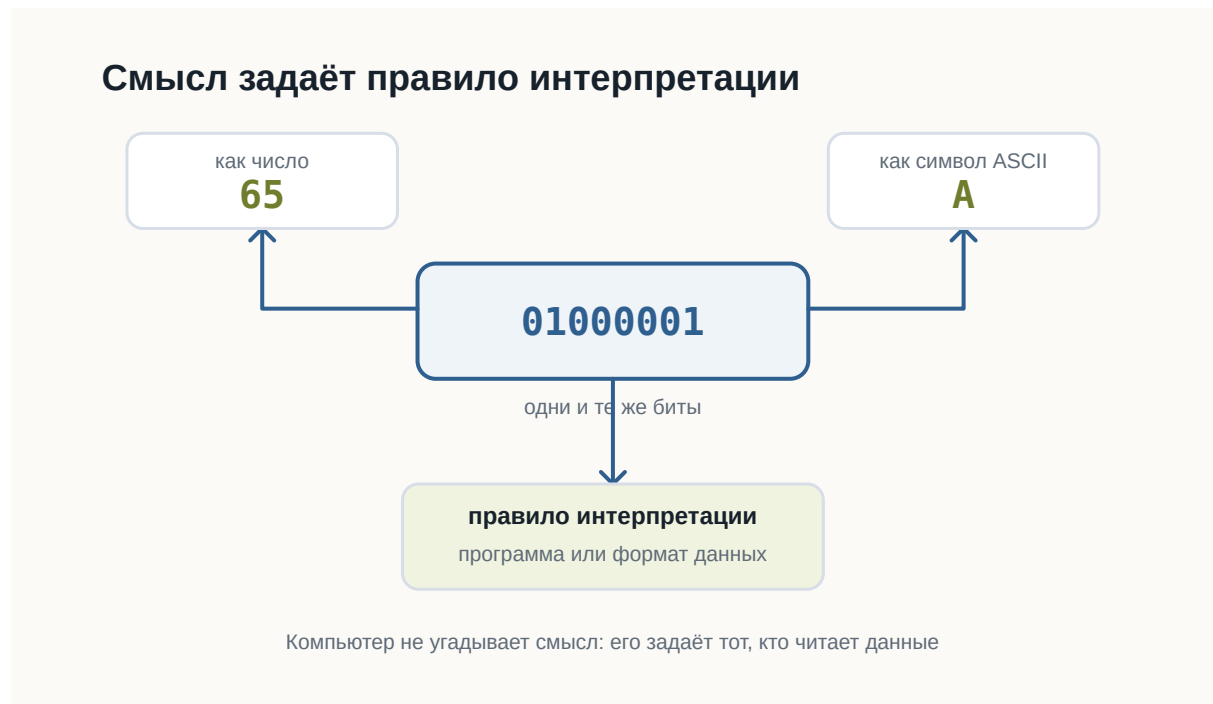


Рисунок 1.7: Одинаковые биты получают смысл только через правило интерпретации.

Как хранится текст

Компьютер не хранит буквы.

Он хранит номера букв.

Например,

Символ	Код
A	65
B	66
C	67

Когда вы набираете

ABC

в памяти оказывается примерно следующее:

65 66 67

или в двоичном виде

01000001

01000010

01000011

Позже программа снова превращает эти числа в буквы.

Компьютер хранит не буквы, а их коды

Символ → код → биты

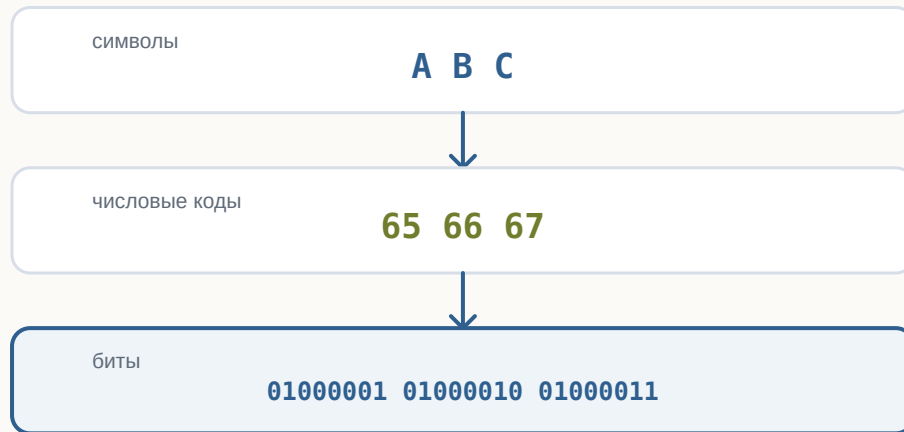


Рисунок 1.8: Текст хранится как числовые коды, представленные битами.

Почему этого оказалось недостаточно

ASCII содержал только 128 символов.

Для английского языка этого хватало.

Но как быть с

Привет

□□□□□

□□□□□

□

Появился Unicode.

Он назначает номер практически каждому символу, существующему сегодня.

Поэтому современные программы способны одновременно показывать русский, японский, арабский и эмодзи.

Как хранится число

Компьютер не знает, что такое “сорок два”.

Он хранит двоичное представление.

Например,

42

становится

00101010

Для компьютера это просто набор битов.

Если программа знает, что это число, она выполняет арифметику.

Если считает, что это символ, получит совершенно другой результат.

Как хранится изображение

Фотография — это вовсе не рисунок.

Это огромная таблица пикселей.

Например,

□ □ □
□ □ □

Каждый пиксель имеет цвет.

Цвет тоже можно записать числами.

Например,

Красный

$R = 255$

$G = 0$

$B = 0$

или

Зелёный

$R = 0$

$G = 255$

$B = 0$

Таким образом фотография превращается в огромное количество чисел.

А числа уже легко представить битами.

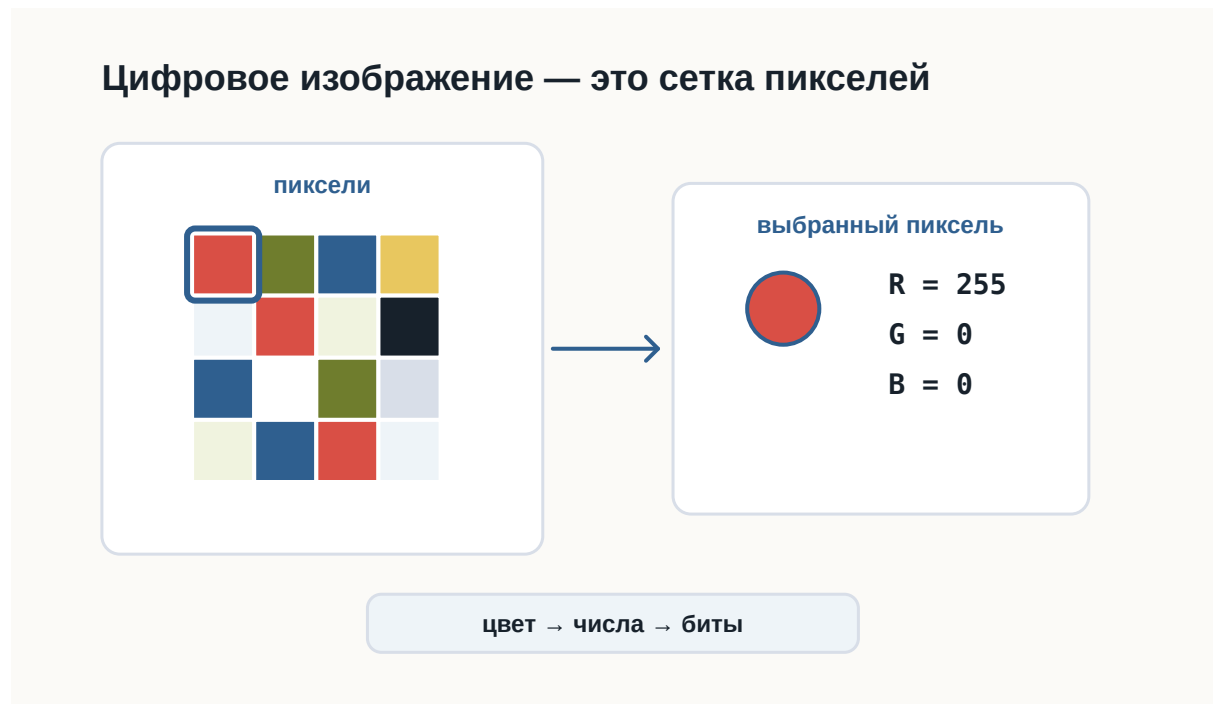


Рисунок 1.9: Изображение хранится как сетка пикселей, а цвет каждого пикселя задаётся числами RGB.

Как хранится музыка

Музыка — это изменение давления воздуха.

Микрофон много тысяч раз в секунду измеряет это давление.

Получается последовательность чисел.

Например,

125

127

130

126

120

...

Компьютер хранит именно эти числа.

При воспроизведении динамик выполняет процесс в обратную сторону.

Как хранится программа

Самое интересное — программы тоже являются данными.

Файл Python

```
print("Hello")
```

хранится как обычный текст.

Но после запуска интерпретатор читает этот текст и начинает выполнять инструкции.

То есть один и тот же набор битов может быть:

- текстом;
- программой;
- изображением;
- музыкой.

Все определяется тем, **кто именно читает эти данные.**

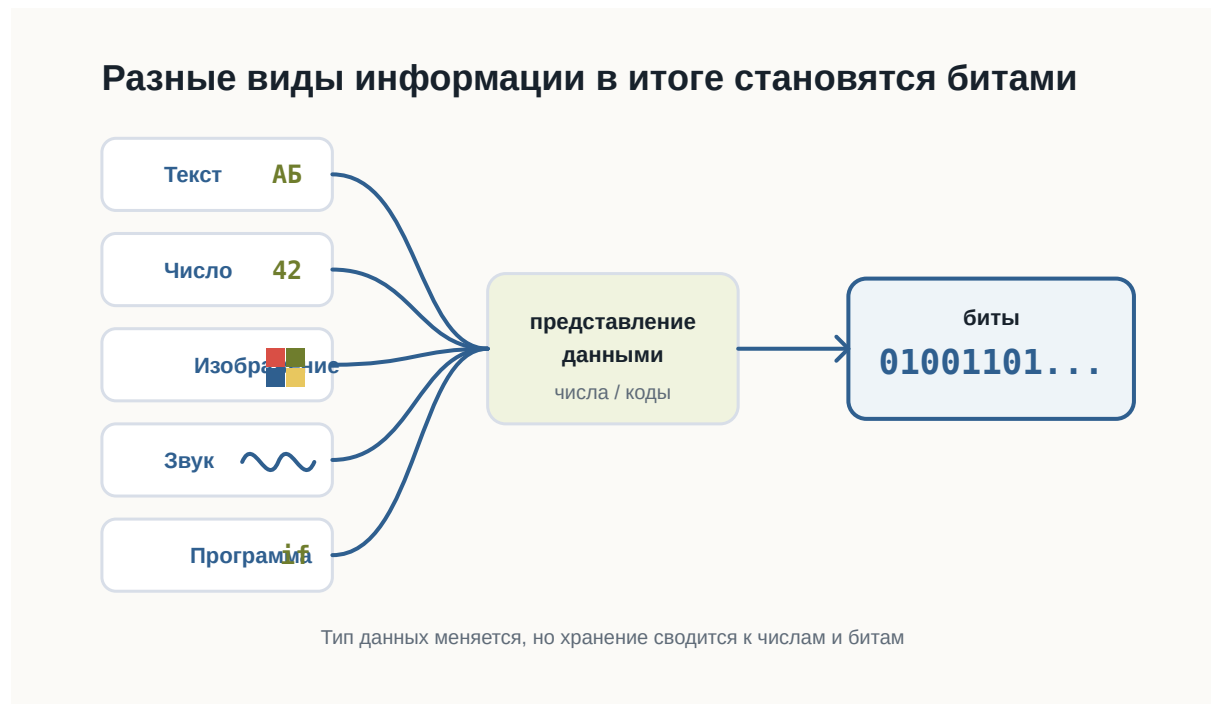


Рисунок 1.10: Текст, числа, изображения, звук и программы сводятся к числам и битам.

Главное, что нужно запомнить

Компьютер хранит только последовательности битов.

Никаких букв, фотографий или музыки внутри памяти нет.

Есть лишь числа.

Именно программы знают, как превратить эти числа обратно в текст, изображения, звук или команды.

Поэтому можно сказать так:

Компьютер хранит не сами данные, а их двоичное представление.

Что дальше?

Теперь мы понимаем:

- почему всё сводится к битам;
- как появляются байты;
- почему существуют кодировки;
- как текст, изображения и числа становятся последовательностями битов.

Но остается следующий вопрос.

Даже если программа уже хранится в памяти как набор битов, **как процессор начинает её выполнять?**

Именно об этом будет следующая глава:

«Как компьютер выполняет программы».

2.4 Как запускаются программы

Когда вы запускаете программу, компьютер не «читает намерения».

Он выполняет конкретную последовательность действий: находит файл, передает его нужному инструменту, выделяет ресурсы и выполняет инструкции.

После этой главы вы сможете

- объяснить, что происходит при запуске программы;
- понимать роль файла с кодом;
- отличать программу от процесса;
- понимать, зачем нужен интерпретатор Python.

Файл с кодом

Python-программа обычно хранится в файле с расширением `.py`.

Например:

```
hello.py
```

Внутри файла находится текст программы.

Сам файл еще не выполняется. Чтобы код начал работать, его должен обработать Python.

Интерпретатор

Интерпретатор Python - это программа, которая читает Python-код и выполняет его.

Когда вы запускаете команду:


```
python hello.py
```

вы просите интерпретатор Python прочитать файл `hello.py` и выполнить инструкции внутри него.

Процесс

Во время запуска операционная система создает процесс.

Процесс - это работающая программа вместе с ресурсами, которые ей выделены.

Например, процесс может использовать:

- память;
- файлы;
- ввод с клавиатуры;
- вывод в терминал.

i Простая схема

Файл с кодом передается интерпретатору, интерпретатор выполняет инструкции, операционная система управляет ресурсами.

Короткий итог

Программа хранится как текстовый файл.

При запуске Python читает этот файл и выполняет инструкции.

Работающая программа становится процессом.

Практика

Объясните своими словами, чем отличается файл `hello.py` от запущенной программы.

2.5 Зачем нужны языки программирования

Компьютер в итоге работает с низкоуровневыми инструкциями.

Но человеку неудобно писать большие программы в виде нулей, единиц и машинных команд.

Поэтому появились языки программирования.

После этой главы вы сможете

- объяснить, зачем нужны языки программирования;
- понимать, почему Python-код должен быть переведен в действия компьютера;
- отличать удобство для человека от формы, удобной для машины;
- понимать роль синтаксиса.

Язык как мост

Язык программирования помогает человеку описать решение в форме, которую можно обработать автоматически.

Например, человеку понятнее написать:

```
print("Hello")
```

чем вручную описывать низкоуровневые операции, которые нужны для вывода текста на экран.

Синтаксис

У каждого языка есть правила записи.

Эти правила называются синтаксисом.

Если нарушить синтаксис, компьютер не сможет правильно разобрать программу.

 Типичная ошибка

Новички часто думают, что синтаксис - это формальность. На самом деле это договоренность, по которой инструмент понимает ваш код.

Почему языков много

Разные языки создавались для разных задач.

Одни удобны для системного программирования, другие - для анализа данных, сайтов, автоматизации или обучения.

Нет одного идеального языка для всех случаев.

Короткий итог

Язык программирования нужен, чтобы человек мог описывать решения понятнее и быстрее.

Компьютер все равно выполняет точные инструкции, но язык помогает записывать эти инструкции на более удобном уровне.

Практика

Возьмите любую бытовую задачу и попробуйте записать ее как набор команд с очень строгими правилами.

2.6 Почему Python

Python часто рекомендуют новичкам не потому, что он самый простой во всем. Его выбирают потому, что он позволяет быстро перейти от идеи к работающему коду и при этом используется в реальных задачах.

После этой главы вы сможете

- объяснить, почему Python подходит для старта;
- понимать, где Python применяется на практике;
- видеть сильные и слабые стороны Python;
- понимать, что изучение Python - это не только синтаксис.

Читаемость

Python-код обычно близок к обычному описанию действий.

Например:

```
name = "Vlad"
print(name)
```

В этом примере можно догадаться, что программа сохраняет имя и выводит его.

Где используется Python

Python применяют в разных областях:

- backend-разработка;
- автоматизация;
- анализ данных;
- машинное обучение;
- тестирование;
- скрипты для рабочих задач.

Для начинающего это удобно: один язык открывает доступ к разным направлениям.

Ограничения

Python не всегда лучший выбор.

Например, для некоторых задач важна максимальная скорость выполнения или низкоуровневый контроль над памятью.

Но для обучения, прототипирования и множества рабочих задач Python подходит очень хорошо.

Главный плюс Python

Python помогает быстрее увидеть результат и сосредоточиться на логике программы.

Короткий итог

Python хорош для старта, потому что он читаемый, практичный и широко используется.

Но важно учить не только команды языка, а общий подход к решению задач.

Практика

Найдите три задачи из своей жизни или работы, которые можно было бы автоматизировать с помощью Python.

2.7 Рабочее пространство Python

Перед тем как писать программы, нужно подготовить рабочее пространство.

Рабочее пространство - это набор инструментов и папок, в которых вы будете писать, запускать и хранить код.

После этой главы вы сможете

- понимать, зачем нужен редактор кода;
- понимать роль терминала;
- создать папку проекта;
- объяснить, зачем нужно виртуальное окружение.

Основные инструменты

Для начала нужны:

- установленный Python;
- редактор кода;
- терминал;
- папка проекта.

Этого достаточно, чтобы написать и запустить первую программу.

Папка проекта

Каждый проект лучше хранить в отдельной папке.

Например:

```
python-start/
```

Внутри будут лежать файлы программы, заметки, настройки и позже зависимости.

Терминал

Терминал позволяет запускать команды.

Например:

```
python --version
```

Эта команда показывает установленную версию Python.

Виртуальное окружение

Позже у разных проектов могут появиться разные зависимости.

Чтобы они не мешали друг другу, используют виртуальные окружения.

i Пока достаточно понимать идею

На первом шаге важно знать, что виртуальное окружение изолирует зависимости проекта. Подробно мы разберем это позже.

Короткий итог

Рабочее пространство помогает держать код в порядке.

Редактор нужен для написания кода, терминал - для запуска команд, папка проекта - для организации файлов.

Практика

Создайте папку `python-start` и проверьте в терминале версию Python командой `python --version`.

2.8 Первая Python-программа

Теперь можно написать первую программу.

Она будет очень короткой, но в ней уже есть все важные элементы: файл, код, запуск и результат.

После этой главы вы сможете

- создать файл Python-программы;
- написать команду вывода текста;
- запустить программу из терминала;
- понять, где появляется результат.

Создаем файл

Создайте файл:

```
hello.py
```

Расширение `.py` показывает, что это файл с Python-кодом.

Пишем код

Добавьте в файл одну строку:

```
print("Привет, мир!")
```

Команда `print()` выводит текст.

Текст внутри кавычек называется строкой.

Запускаем программу

Откройте терминал в папке проекта и выполните:

```
python hello.py
```

Если все настроено правильно, вы увидите:

```
Привет, мир!
```

! Что произошло

Python прочитал файл `hello.py`, выполнил команду `print()` и вывел текст в терминал.

Если команда не сработала

Возможные причины:

- Python не установлен;
- терминал открыт не в той папке;
- файл называется не `hello.py`;
- в коде есть опечатка.

Ошибки на этом этапе нормальны. Важно научиться читать сообщение и проверять шаги по порядку.

Короткий итог

Первая программа показывает полный цикл работы:

```
файл -> код -> запуск -> результат
```

Даже простая строка `print("Привет, мир!")` уже является полноценной программой.

Практика

Измените текст внутри `print()` и запустите программу еще раз.